



TIR-Judge: ก้าวข้ามขีดจำกัดของ LLM Judge ด้วย Tool-Integrated Reasoning

อนาคตของการประเมิน AI ที่ผสาน “ความเข้าใจภาษา” เข้ากับ
“ความแม่นยำของการรันโค้ด” (Based on Google Research)

LLM เป็นกรรมการที่ดี แต่ยังมี ‘จุดบอด’

สิ่งที่ LLM ทำได้ดี



ประเมินความสละสลวยของข้อความ

วิเคราะห์บริบททางภาษา

จัดหมวดหมู่เนื้อหา

จุดบอดของ Intrinsic Text-based Reasoning

$$\int_0^\infty e^{-x^2} dx = \sqrt{\pi}$$
$$\sum_{n=0}^\infty \frac{(-1)^n}{2n+1} = \frac{\pi}{4}$$

$$\text{len}(\text{"สวัสดี"}) == \underline{\underline{3}}$$

การนับคำหรืออักขระที่แม่นยำ 100%

การคำนวณคณิตศาสตร์ที่ซับซ้อน

การตรวจสอบข้อเท็จจริงด้วยตรรกะเชิงสัญลักษณ์

LLM Judge ปัจจุบันพึ่งพาเพียงการให้เหตุผลผ่านตัวอักษร (Text-based)
ทำให้ไม่สามารถตรวจสอบเงื่อนไขที่ต้องการความแม่นยำสมบูรณ์ได้

วิวัฒนาการของระบบประเมิน AI

Gen 1: Scalar Reward Models

กลไก:

ให้คะแนนผลลัพธ์โดยตรง

จุดอ่อน:

ขาดความโปร่งใส
ไม่มีเหตุผลประกอบ



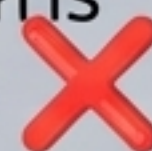
Gen 2: Text-based Reasoning Judges

กลไก:

คิดก่อนให้คะแนนด้วย
Chain-of-Thought (CoT)

จุดอ่อน:

อาศัยการคาดเดาความน่า
จะเป็นทางภาษา
มักพลาดเรื่องตรรกะและการ
คำนวณ



Gen 3: TIR-Judge
(The Breakthrough)

กลไก:

คิด + เขียนโค้ด + รันใน
Sandbox เพื่อตรวจสอบ

จุดแข็ง:

ตัดสินใจบนพื้นฐานของ
"ความจริงที่พิสูจน์ได้"
(Executable Evidence)
แม่นยำและตรวจสอบได้จริง



ให้กรรมการถือ 'เครื่องคิดเลข' แทนการคาดเดา

คำสั่ง: เขียนบทกวีความยาวอย่างน้อย 350 คำ

โมเดลตอบกลับมาเป็นบทกวี พร้อมบอกว่า 'The response contains 392 words.'



Text-based Judge

หลงเชื่อข้อความที่โมเดลสร้างขึ้น
และให้คะแนน 'ผ่าน' (Pass)



TIR-Judge

เขียน Python script
`len(response.split())`
รันนับคำจริง พบว่ามีแค่ 321 คำ
จึงตัดสินใจให้ 'ไม่ผ่าน' (Fail)

TIR-Judge ตัดสินใจโดยอิงจาก
'ความจริงที่รันได้' (Executable Evidence)

3 เสาหลักของการสร้าง TIR-Judge



Task Diversity (ความหลากหลาย ของงาน)

เรียนรู้การประเมินทั้งงาน
ที่ใช้โค้ดตรวจสอบได้
(Math/Coding) และงาน
ที่ต้องใช้ดุลยพินิจ
(Safety/Helpfulness)



Judgment Flexibility (ความยืดหยุ่นในการ ตัดสิน)

รองรับทุกกระบวนการ
ประเมิน:

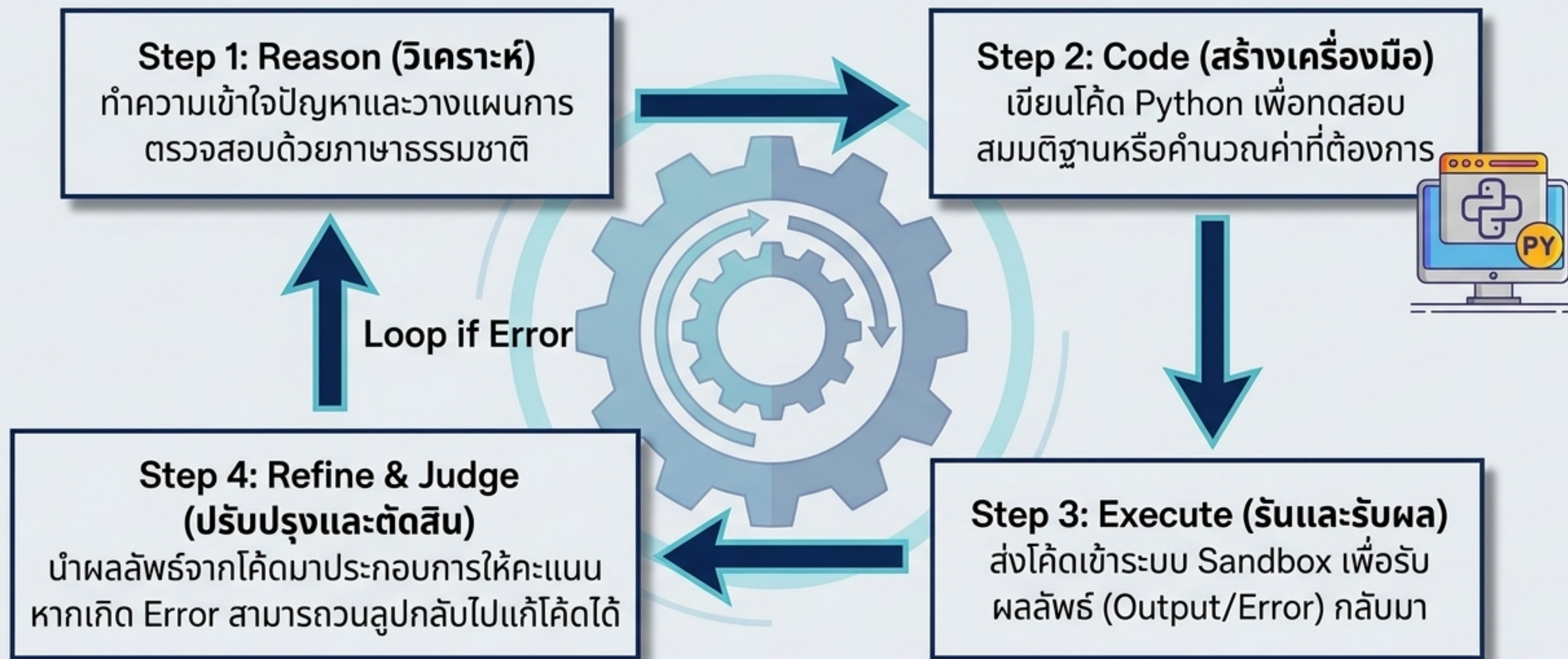
- Pointwise (ให้คะแนนเดียว)
- Pairwise (จับคู่เวลาหาผู้ชนะ)
- Listwise (จัดอันดับจากกลุ่ม)



Iterative RL (การเรียนรู้และพัฒนา ด้วยตัวเอง)

ใช้ Reinforcement
Learning เพื่อพัฒนากลไก
การใช้ Tool ได้ด้วยตัวเองโดย
ไม่ต้องพึ่งพาข้อมูล
Distillation

กายวิภาคของการคิดแบบ Tool-Integrated Reasoning



กระบวนการนี้ถูกฝึกสอนแบบ End-to-end ผ่าน Reinforcement Learning (RL)

กฎหลักของการฝึกสอน: ออกแบบ 'รางวัล' ให้ฉลาดและเป๊ะ



โมเดลจะได้รับคะแนนเต็ม (Full Credit) ก็ต่อเมื่อทำสำเร็จครบทั้ง 3 เงื่อนไขเท่านั้น ป้องกันการเดาถูกแต่ใช้*วิธีการผิด

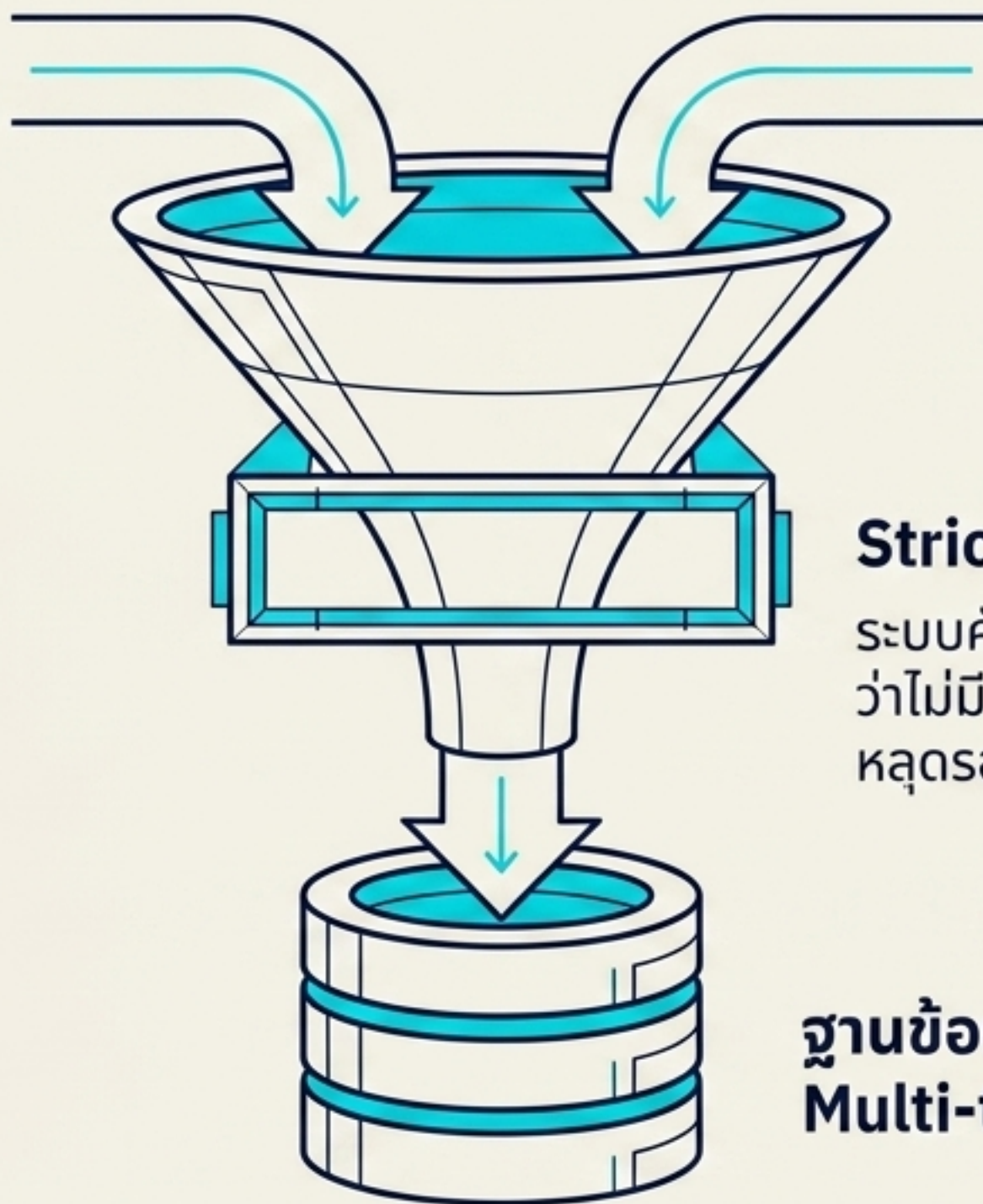
เครื่องยนต์ต้องการเชื้อเพลิง: ข้อมูลการฝึกสอนที่หลากหลาย

Real Preference Pairs (ข้อมูลจากมนุษย์)

ข้อมูลด้าน **Helpfulness, Safety,**
และ **Code** จากชุดข้อมูลมาตรฐาน

Synthetic Preference Pairs (ข้อมูลสังเคราะห์)

คำตอบที่สร้างจาก **Open-source LLMs**
แล้วนำมาตรวจสอบความถูกต้องด้วยโค้ด
(Verifiable functions)

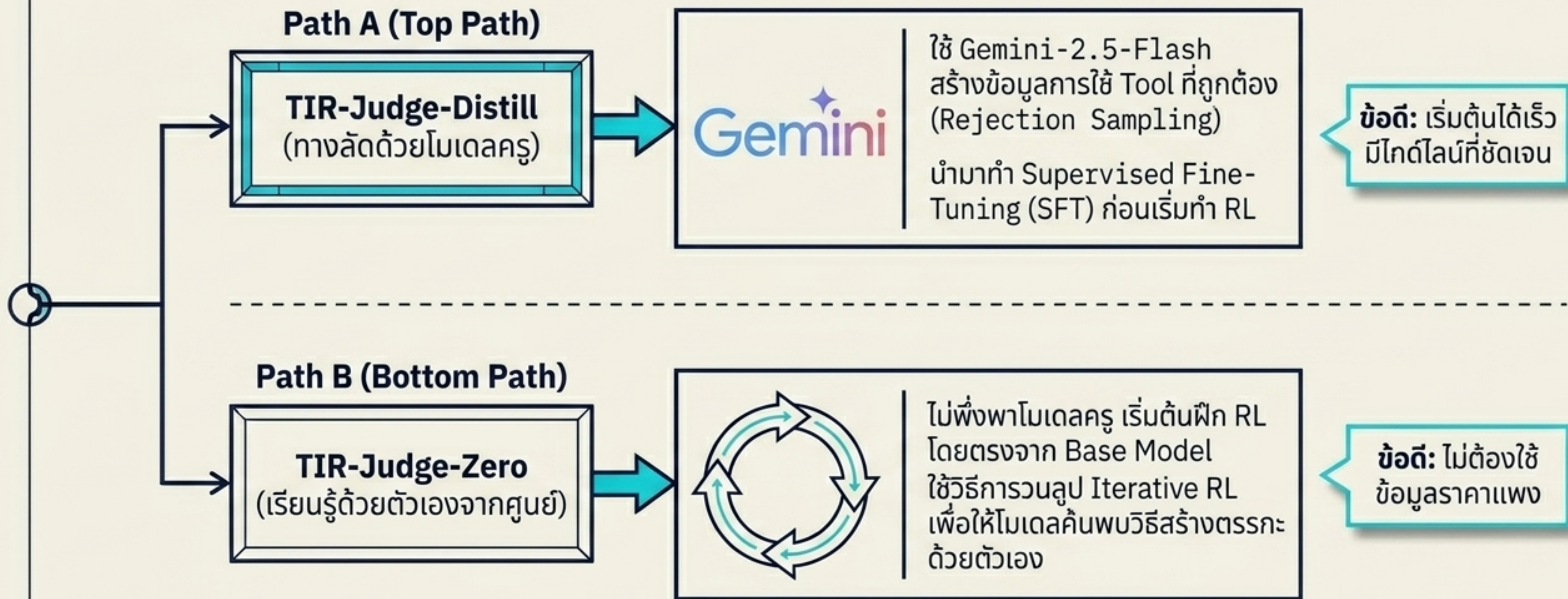


Strict 8-gram Decontamination

ระบบคัดกรองขึ้นเด็ดขาด เพื่อให้มั่นใจ
ว่าไม่มีข้อมูลใดๆ ในชุดฝึกสอน (26k Pairs)
หลุดรอดไปตรงกับชุดทดสอบ

ฐานข้อมูลที่พร้อมสำหรับการทำ
Multi-turn RL

2 เส้นทางสู่ความสำเร็จ: Distill vs Zero



ปาทหาริย์ของ TIR-Judge-Zero (Self-Evolving AI)

1. Reinforcement Learning (RL)

เทรนโมเดลด้วยตัวเองในรอบแรก

2. Rejection Sampling (RS)

สุ่มสร้างคำตอบ เลือกเฉพาะอันที่
“ตอบถูก + โค้ดรันผ่าน + สั้นที่สุด”

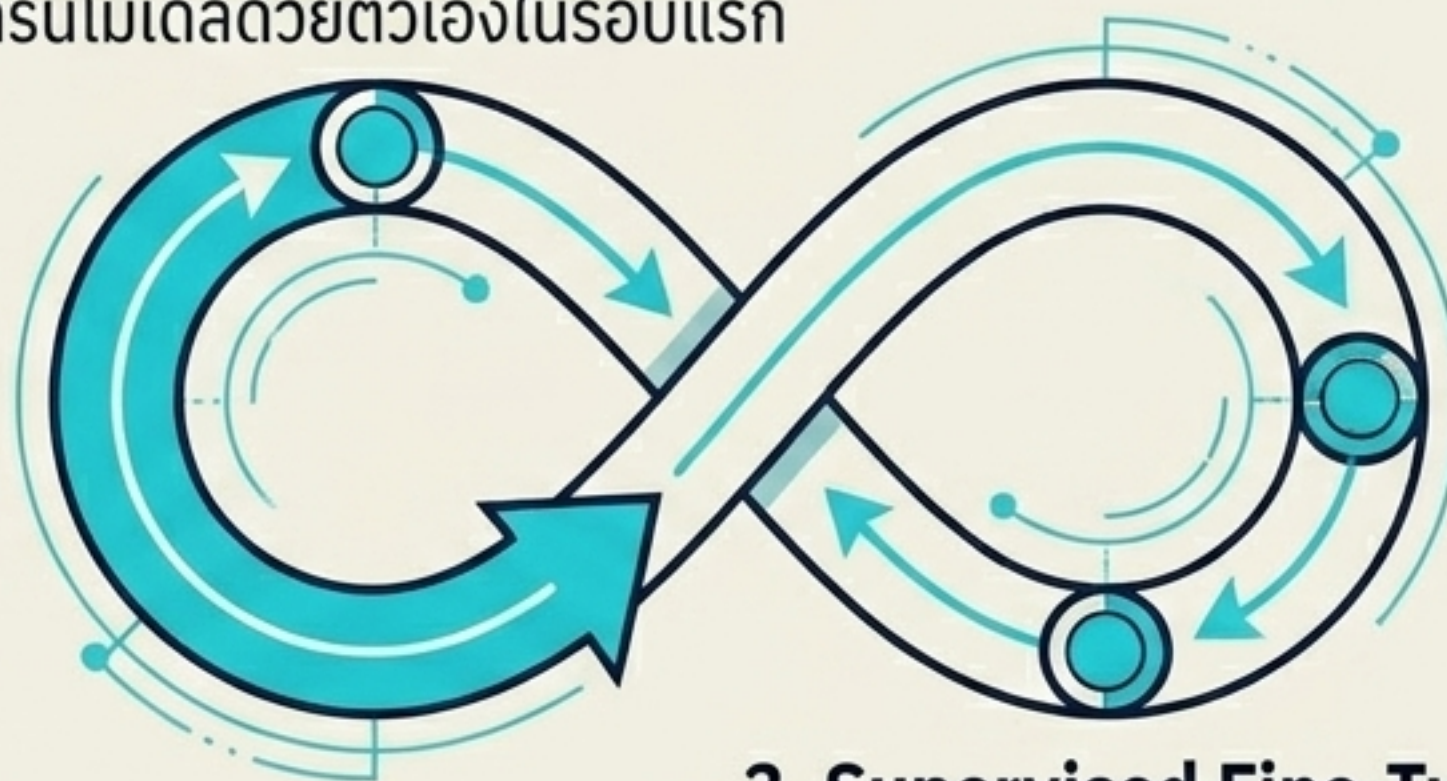
✓ + `</>` + ✓

3. Supervised Fine-Tuning (SFT)

นำคำตอบที่ดีที่สุดกลับมาเป็นตำราสอนตัวเองใหม่

4. Iterate (วนซ้ำ)

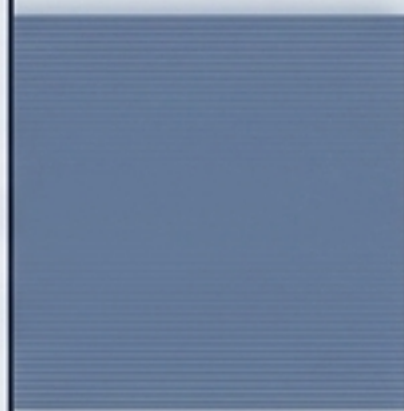
นำโมเดลที่ฉลาดขึ้นไปทำ RL
ในรอบถัดไป



TIR-Judge-Zero สามารถเอาชนะเวอร์ชันที่ใช้ข้อมูลจากโมเดลครู (Distill) ได้
พิสูจน์ว่า AI สามารถ ‘ตรัสรู้’ วิธีการใช้เครื่องมือได้อย่างมีประสิทธิภาพด้วยตัวเอง

บทพิสูจน์เชิงประจักษ์: ขนาดลอยในสมรภูมิ Reasoning

Text-based Judge



TIR-Judge



ชนะโมเดลคู่แข่งในพิกัดเดียวกัน

เอาชนะ Text-based Judge ในรุ่นน้ำหนักเท่ากัน (8B) สูงสุดถึง **+6.4%** (Pointwise) และ **+7.7%** (Pairwise).

API อย่างเดียวไม่พอ ต้องมี RL

การนำโมเดลปกติมาต่อ API ให้ใช้ Tool แทนไม่ช่วยเพิ่มความแม่นยำ โมเดลต้องผ่านกระบวนการฝึก RL ของ ~~โมเดล~~ ผ่านกระบวนการตีตรา: ของ TIR-Judge เท่านั้น จึงจะรู้ว่า 'เมื่อไหร่ควรเขียนโค้ด'.

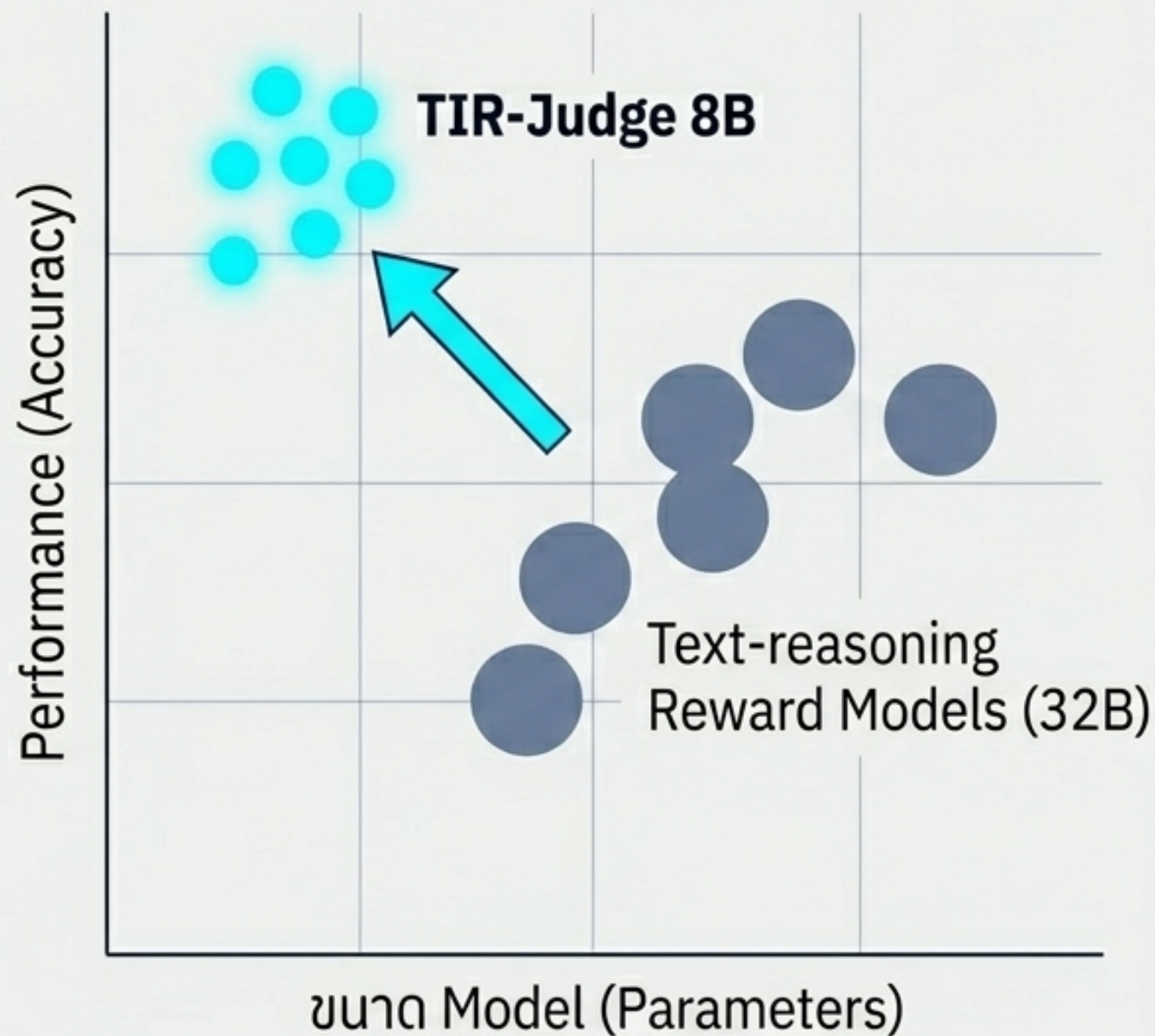
The Listwise Challenge: เล็กพริกขี้หนู

การประเมินแบบ Listwise (จัดอันดับคำตอบหลายข้อพร้อมกัน) คือโจทย์ที่หินที่สุดสำหรับ LLM Judge

Leaderboard Table		
1	Claude-Opus-4	76.5% (Frontier Model)
2	Gemini-1.5-Pro	75.9%
3	TIR-Judge-Zero 8B	73.4%
4	Claude-Sonnet-4	69.5%

ด้วยขนาดเพียง 8B Parameters TIR-Judge-Zero ทำผลงานได้เทียบเท่ากับ 96% ของสมรรถนะระดับ Claude-Opus-4 ในการประเมินที่ซับซ้อนที่สุด

ทำไมต้องจ่ายแพง ในเมื่อ 8B ก็ทำได้?



Observation:

TIR-Judge ขนาด 8B
TIR-Judge ขนาด 8B สามารถเอาชนะ
Text-reasoning Reward Models ขนาด 32B
(ใหญ่กว่า 4 เท่า) ได้อย่างสบายๆ บน PPE Dataset

ยุคเดิม - Compute-heavy:

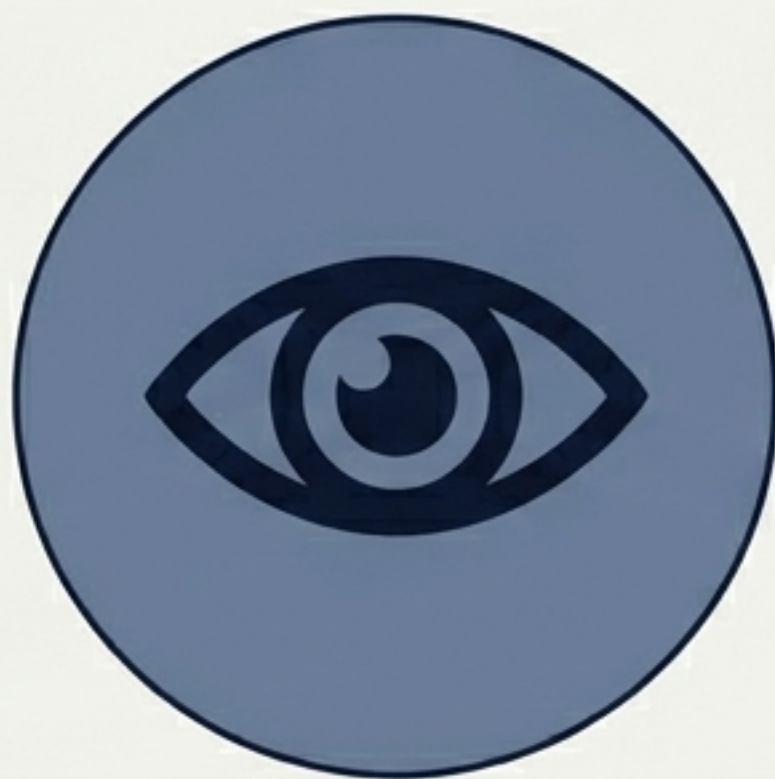
เพิ่มขนาดพารามิเตอร์เพื่อให้โมเดล 'จำ' ได้เก่งขึ้น.

ยุคใหม่ - Tool-Integrated:

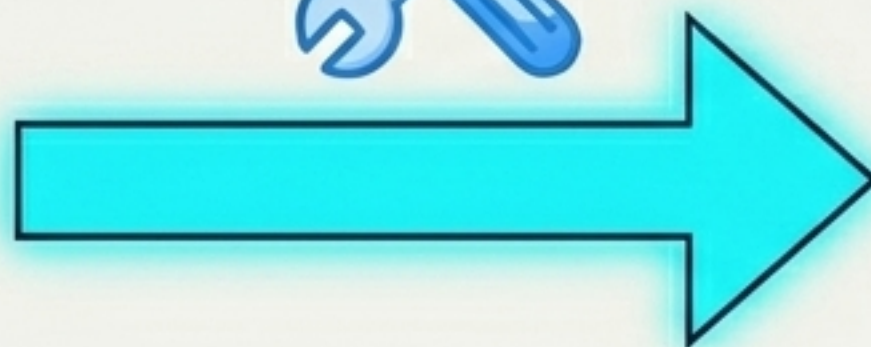
มอบ 'เครื่องมือ' และสอนตรรกะการเรียกใช้
ให้โมเดลขนาดเล็กพิสูจน์ความจริงด้วยตัวเอง.

'วิธีการคิดที่ถูกต้อง' สำคัญกว่า 'ขนาดของสมอง'

จาก “ผู้ประเมิน” สู่ “Agentic Verifier”



การอ่านและให้คะแนนความน่าจะเป็น
(Passive Evaluation)



การลงมือปฏิบัติและพิสูจน์ความจริงด้วยโค้ด
(Agentic Verification)

เราไม่สามารถก้าวไปสู่ AGI ได้ หากปราศจากระบบประเมินผลที่แม่นยำ
TIR-Judge เปลี่ยนกระบวนการตัดสินจากการเดาเป็นการพิสูจน์
ปลดล็อกการประเมินและอัปเดตตัวเองได้อย่างไร้ขีดจำกัดด้วย Zero-distillation

บทสรุป (Key Takeaways)

Tool-Integration is Essential

การประเมินตรรกะที่แม่นยำ
ไม่สามารถทำได้ด้วยภาษา
ธรรมชาติเพียงอย่างเดียว

LLM Judge ในอนาคตต้องมี
Execution Environment
ในตัว

RL Unlocks the Power

การแค่ต่อ API ไม่เกิดประโยชน์
โมเดลต้องผ่านกระบวนการ
Multi-turn RL เพื่อเรียนรู้ว่า
“ควรใช้ Tool เมื่อไหร่
และใช้อย่างไร”.

Self-Evolution is Real

ปาฏิหาริย์ของ TIR-Judge-
Zero พิสูจน์ว่า โมเดลขนาดเล็ก
สามารถเรียนรู้ตรรกะและ
พัฒนาทักษะได้ด้วยตัวเอง
ผ่าน Iterative RL โดยไม่ต้อง
ใช้ข้อมูลสอนจากคนหรือ AI
ยักษ์ใหญ่